Atividade Avaliativa 3

Instruções do exercício

Catálogo de Produtos com Web Scraping

Valor: 2,5 pontos
Tecnologias: Python, Flask, HTML, Web Scraping
Site utilizado: https://scrapingsandbox.com/
Equipes: Até 4 alunos.

1. Objetivo

Desenvolver uma aplicação Web utilizando **Flask** capaz de coletar informações de produtos disponíveis no site Scraping Sandbox por meio de **Web Scraping**.
A aplicação deverá permitir que o usuário escolha uma **categoria de produtos**. O programa deverá realizar a coleta dos dados do site, identificar a categoria dos produtos e apresentar somente aqueles pertencentes à categoria escolhida pelo usuário.
Importante: a seleção e a filtragem da categoria deverão ser realizadas pela **aplicação desenvolvida pelo aluno**. Não é necessário automatizar cliques, filtros ou outros recursos interativos existentes no site Scraping Sandbox.

2. Funcionamento esperado

O sistema deverá seguir, de forma geral, o seguinte fluxo:

Usuário acessa a aplicação Flask
↓
Escolhe uma categoria
↓
Clica em Pesquisar
↓
Flask recebe a categoria escolhida
↓
Programa acessa o Scraping Sandbox
↓
Web Scraping coleta os produtos
↓
Python identifica e filtra a categoria
↓
Programa realiza as análises
↓
Flask envia os resultados para o HTML
↓
Produtos são apresentados no navegador

3. Requisitos do sistema

A aplicação deverá atender aos seguintes requisitos:

1. Página inicial

- Apresentar o título da aplicação;
- Permitir que o usuário escolha uma categoria;
- Possuir um botão para realizar a pesquisa.

2. Web Scraping

- Utilizar requests para acessar as páginas;
- Utilizar BeautifulSoup para interpretar o HTML;
- Os dados dos produtos deverão ser obtidos automaticamente do Scraping Sandbox;
- Não será permitido cadastrar manualmente os produtos no código.

3. Dados dos produtos

Para cada produto coletado, deverão ser obtidas e apresentadas, quando disponíveis no site:

- nome;
- preço;
- categoria;
- avaliação;
- link do produto.

4. Filtragem por categoria

- A categoria deverá ser escolhida pelo usuário na aplicação Flask;
- O programa deverá identificar a categoria dos produtos coletados;
- Após a coleta, o próprio programa Python deverá filtrar os produtos;
- Na página de resultados deverão aparecer somente os produtos correspondentes à categoria escolhida.

Por exemplo, conceitualmente:

```
if categoria_produto == categoria_escolhida:
    produtos_filtrados.append(produto)
```

Esse código é apenas uma representação da lógica esperada. Cada grupo deverá desenvolver sua própria solução.

5. Análise dos produtos

A aplicação deverá calcular e apresentar, considerando **somente os produtos da categoria selecionada**:

- quantidade de produtos encontrados;
- produto de menor preço;
- produto de maior preço;
- preço médio dos produtos;
- produto com melhor avaliação, quando essa informação estiver disponível.

Caso mais de um produto possua o mesmo menor preço, maior preço ou melhor avaliação, a aplicação poderá apresentar um deles ou todos os produtos empatados.

6. Apresentação dos resultados

Os produtos deverão ser apresentados em uma página HTML utilizando **Jinja**, podendo ser organizados em tabela, lista ou cards.

A página deverá apresentar claramente:

Categoria pesquisada

Quantidade de produtos encontrados

Menor preço
Maior preço
Preço médio
Melhor avaliação

Produto 1
Nome:
Preço:
Categoria:
Avaliação:
Link:

Produto 2
...

Não será avaliada a criação de um design sofisticado. O mais importante é a **organização e o funcionamento da aplicação**.

4. Uso obrigatório do Flask

O projeto deverá utilizar Flask e apresentar comunicação entre o formulário HTML e o programa Python.

Flask
@app.route()
render_template()
request

O fluxo deverá ser:

Formulário HTML
↓
Flask
↓
Python

↓
Web Scraping
↓
Processamento
↓
Flask
↓
HTML / Jinja

Não será aceito um programa que apresente os resultados **somente no terminal**.

5. Paginação

O Scraping Sandbox apresenta produtos distribuídos em diferentes páginas.

O programa deverá realizar o Web Scraping em **mais de uma página**, demonstrando o uso de paginação.

Não é obrigatório coletar todos os produtos existentes no site. Entretanto, a quantidade de páginas utilizada deverá ser suficiente para que a categoria selecionada possua dados que permitam realizar as análises solicitadas.

6. Restrições

Para este trabalho:

- não é necessário utilizar Selenium ou Playwright;
- não é necessário automatizar cliques no site;
- não é necessário executar JavaScript do Scraping Sandbox;
- não é obrigatório utilizar banco de dados;
- não é necessário utilizar Pandas;
- os produtos não poderão ser cadastrados manualmente no código.

O foco da atividade é aplicar os conteúdos estudados de **Flask + Web Scraping + processamento de dados com Python**.

7. Entrega

O grupo deverá entregar:

- arquivo(s) Python;
- pasta templates com os arquivos HTML;
- demais arquivos necessários para executar a aplicação.

O projeto entregue deverá estar completo e ser possível executá-lo para demonstrar seu funcionamento.

8. Apresentação

Durante a apresentação, o grupo deverá:

1. executar a aplicação Flask;
2. acessar a aplicação pelo navegador;

3. escolher uma categoria;
4. realizar a pesquisa;
5. mostrar os produtos encontrados;
6. mostrar os cálculos realizados;
7. realizar uma nova pesquisa utilizando outra categoria;
8. explicar resumidamente como o Web Scraping foi implementado.

O professor poderá solicitar que o grupo explique partes específicas do código ou faça uma pequena alteração durante a apresentação.

9. Critérios de avaliação

Critério	**Valor**
Aplicação Flask e formulário para escolha da categoria	**0,40**
Web Scraping e coleta correta dos dados	**0,70**
Filtragem e apresentação dos produtos	**0,40**
Cálculos e análise dos produtos	**0,40**
Organização, funcionamento e comentários do código	**0,30**
Apresentação e demonstração do sistema	**0,30**
Total	**2,50 pontos**

Observação importante

A categoria deverá ser selecionada **dentro da aplicação Flask desenvolvida pelo grupo**. A aplicação deverá coletar os produtos disponíveis no Scraping Sandbox e realizar a filtragem utilizando Python. **Não faz parte deste trabalho automatizar os filtros, botões ou outros recursos interativos do próprio site Scraping Sandbox.**
Essa última observação resolve justamente o problema que identificamos: o aluno não poderá alegar que precisou aprender Selenium porque o filtro do site dependia de JavaScript. O que será avaliado é exatamente o conteúdo que foi trabalhado em sala.

Envio

Clique para adicionar texto

Contagem de palavras: 0

Arrastar os arquivos aqui

Tamanho máximo: 50 MB

ou

Fazer upload de arquivo | Fazer upload a partir da nuvem

Salvar e fechar | Enviar